Gas Usage Forecasting and Delivery Optimization for Scot Forge

Kumar Mantha Vandit Goel Pratyaksh Mathur

April 2026

Abstract

Scot Forge needed a delivery-planning workflow that could forecast gas demand across two Illinois sites and translate that forecast into a nomination plan that respects daily storage activity limits, monthly end-of-month inventory bands, and tiered cash-out penalties. Using 3,410 daily observations from August 2015 through November 2024, we first profiled the three metered usage streams with Polars- and Altair-based exploratory analysis, then benchmarked baseline, tree-based, and deep-learning forecasters. The final production-oriented solution retrains monthly, uses only data available through $t-2$ to forecast day t, blends quantile gradient boosting with linear quantile regression, and sends the resulting P10/P50/P95 demand scenarios into a 60-day rolling linear program solved with SciPy HiGHS. In the 2022–2024 backtest, the optimized policy reduced daily violations from 420 to 156, a 62.9% reduction, while monthly end-of-month violations remained unchanged at 15. Tiered cash-out events also improved materially from 71 to 22 in Tier 1, 47 to 25 in Tier 2, and 302 to 109 in Tier 3.

1 Introduction

Scot Forge is an employee-owned forging manufacturer whose furnace operations consume large amounts of natural gas across a Spring Grove site and a Franklin Park site (Scot Forge, 2026). Recent tariff changes made gas planning materially harder because daily nominations now interact with both daily injection or withdrawal caps and monthly storage-balance requirements. A forecast-only solution was therefore insufficient: even a numerically accurate demand forecast could still produce a bad delivery decision if it created a storage breach or an expensive cash-out event.

The project evolved from a standard time-series assignment into a consulting-style decision support system. The initial ask focused on forecasting next-day usage from meter data, but client discussions and tariff review shifted the operational objective toward penalty-aware delivery optimization. The resulting system combines descriptive EDA, walk-forward forecasting, and constrained optimization so that demand uncertainty is handled explicitly rather than hidden inside a single point estimate.

2 Problem Statement

The source data contains three daily usage streams: Usage 1 from Site 1, Usage 2 from Site 2 meter 1, and Usage 2_1 from Site 2 meter 2. The project objective was twofold: first, forecast total daily gas usage across both sites; second, optimize gas delivery or nomination decisions to minimize daily and monthly tariff penalties and reduce cash-out exposure (Scot Forge, 2026; Northern Illinois Gas Company, 2025). The final operating constraint is critical: current-day usage is not known until the following day, so any production-style policy must forecast day t using information only through day $t-2$.

Three operational hypotheses guided the work. First, the gas series contains exploitable seasonal, weekly, holiday, and lag structure that supports walk-forward prediction. Second, probabilistic forecasts should outperform mean-only forecasts for downstream decision quality because underestimation can drive storage negative and cause withdrawal breaches. Third, a rolling-horizon optimizer with explicit daily and monthly contract limits should reduce daily penalty frequency without destabilizing end-of-month storage compliance. These hypotheses were tested through descriptive EDA and out-of-sample backtesting rather than formal null-hypothesis significance tests, because the implemented repository artifacts focus on forecasting and policy evaluation rather than classical inferential testing.

3 Proposed Methodology

EDA and operational framing. The EDA was implemented as an interactive Marimo application using Polars and Altair. The data spans 3,410 daily records from August 2015 to November 2024. Usage 2 accounts for 84.5% of aggregate consumption, making it the dominant driver of total demand volatility, while Usage 1 contributes 13.6% and Usage 2_1 contributes 2.0%. The EDA also established the core operating facts used later in modeling: total usage peaks in January and February at about 2,099 and 2,034 units, falls to a summer baseline near 1,424 to 1,515 units in July through September, and exhibits a pronounced weekly operating cycle with the lowest average demand on Saturdays. Lag structure reinforced those observations: Usage 1 and Total Usage correlate more strongly with a 7-day lag than a 1-day lag, while Usage 2_1 remains highly persistent even at 30 days.

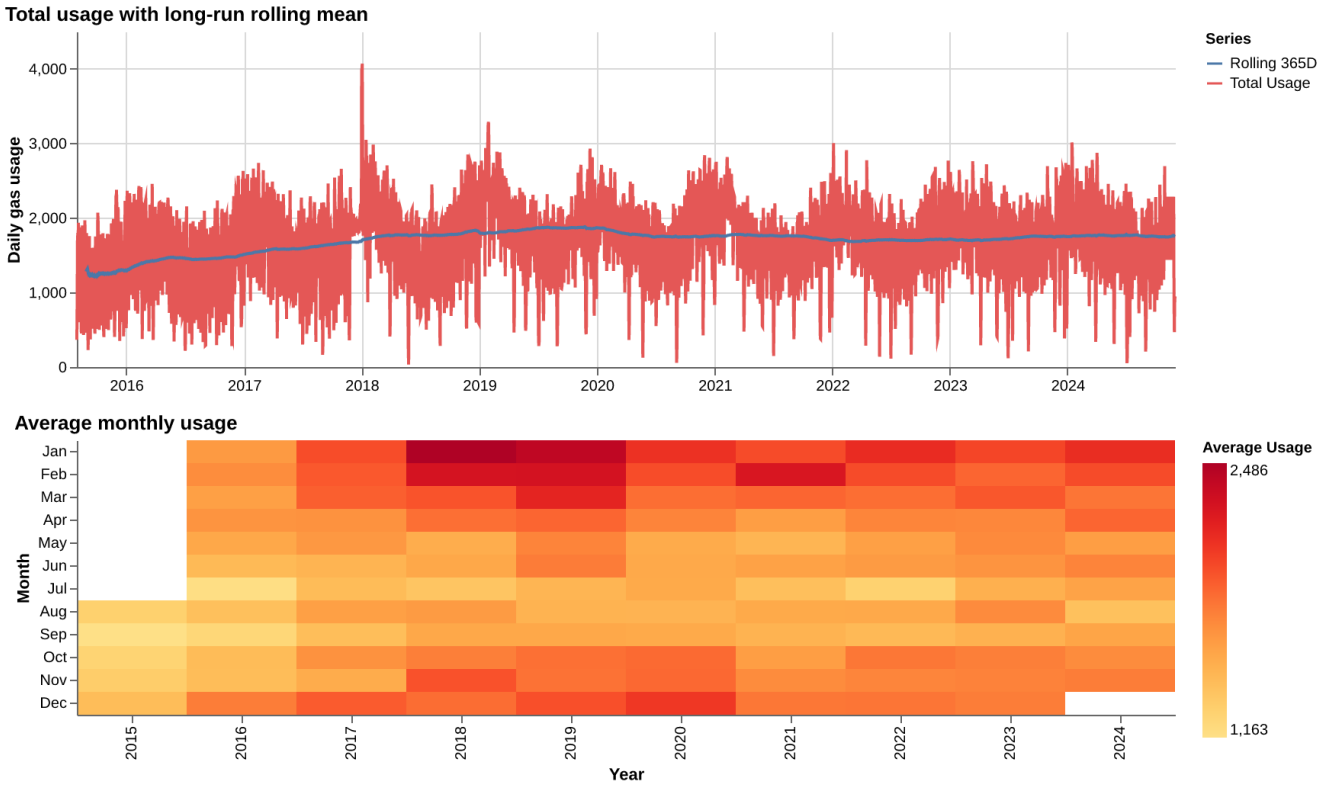

Figure 1: EDA overview used in the main narrative. The time-series view shows long-run seasonality and trend, while the monthly aggregation highlights winter peaks and summer troughs. Additional boxplots, histograms, heatmaps, and diagnostics are moved to the appendix.

Forecast model chronology. During model development, the search space widened deliberately before the final design was fixed. Early work included month-over-month and year-over-year baselines, positive-lag rules, a weighted ensemble baseline, and SARIMAX comparisons. The main benchmark pass then evaluated XGBoost, LightGBM, CatBoost, regime-based variants, residual-tuned variants, an ensemble average, TimesNet, and PatchTST under 1-day walk-forward validation. The best pure forecasting score in that phase came from a regime-plus-residual CatBoost model for Total Usage, but later stakeholder review shifted the objective away from point-forecast accuracy alone and toward operational robustness under tariff rules.

Table 2: Selected model benchmark results from the forecasting comparison phase.

Usage	Model	RMSE	MAE	MAPE	Notes
Total Usage	XGBoost	270.50	203.77	17%	Baseline boosting model
Total Usage	LightGBM	262.14	194.48	19%	Basic LightGBM
Total Usage	CatBoost	258.57	191.82	20%	Best basic tree model
Total Usage	Ensemble	265.16	192.59	21%	Average of XGBoost, LightGBM, CatBoost
Total Usage	CatBoost + regime + residual	258.00	191.77	19%	Best point forecast in benchmark pass
Usage 1	TimesNet	10.71	8.92	3%	Strong facility-level fit
Usage 2	TimesNet calibrated	234.42	193.81	11%	Improved over raw TimesNet
Usage 1	PatchTST	43.42	32.10	25%	Transformer baseline
Usage 2	PatchTST	255.14	184.86	20%	Transformer baseline

Final forecasting pipeline. The final implementation is defined in `forecast_optimize.py`. It engineers calendar, holiday, lag, rolling-window, and storage-state features, then retrains a quantile model at the start of each month for the 2022–2024 backtest. Each quantile is estimated with a nonlinear quantile gradient boosting regressor and a linear quantile regressor solved with HiGHS (Friedman, 2001; scikit-learn developers, n.d.a,n). Their outputs are blended 70:30 in favor of the nonlinear boosting model. The optimizer then uses the conservative P95 usage forecast because earlier experiments with mean prediction left inventory negative and systematically under-nominated delivery on volatile days.

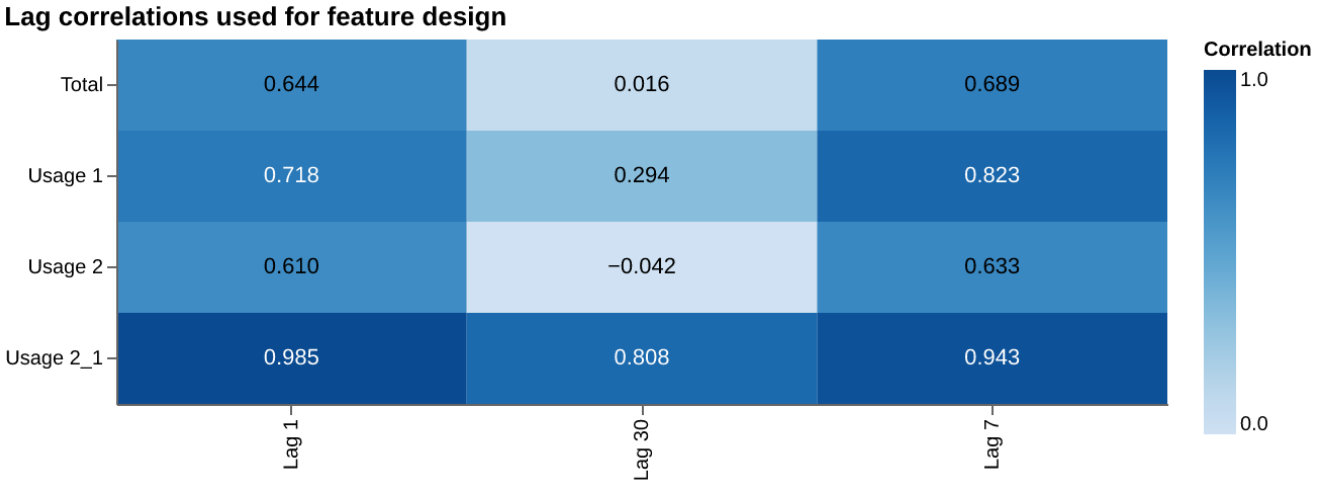

Figure 2: Lag diagnostics used to justify the forecasting features. Seven-day structure dominates Usage 1 and Total Usage, Usage 2 is driven by short-horizon operating conditions, and Usage 2_1 shows slow-moving persistence.

Rolling optimization. The second half of `forecast_optimize.py` converts forecasted usage into a delivery policy. A 60-day linear program is solved with `scipy.optimize.linprog(method="highs")` (SciPy Community, n.d.; Huangfu and Hall, 2018). The objective penalizes aggressive flow while daily constraints enforce month-specific injection and withdrawal caps derived from contract percentages of the 144,841-unit storage capacity. End-of-month bounds also vary by month, from a permissive 0–10% band in March and April to an 85–100% band in October. To make the policy more realistic, the final version adds month-specific injection and withdrawal buffers, month-specific end-of-month buffers, a mid-month steering target, a storage overshoot guard, and fallback solves that relax soft constraints before reverting to the historical delivery. This optimizer was backtested in rolling fashion from January 2022 through September 2024.

4 Analysis and Results

Table 1 summarizes the key backtest outcomes that matter operationally. The optimized policy preserved monthly compliance at the same level as actual operations but materially improved day-to-day execution. Daily violations dropped from 420 to 156, which is a reduction of 264 incidents or 62.9%. Tiered cash-out exposure improved across all penalty levels, with the largest absolute gain in Tier 3, the most expensive category. During development, the first probabilistic optimization prototype cut roughly 150 daily violations, and the final tuned policy improved that gain to 264.

Table 1: Operational KPI summary for the 2022–2024 backtest window.

KPI	Actual	Optimized	Delta	Reduction
Daily violations	420	156	-264	62.9%
Monthly EOM violations	15	15	0	0.0%
Tier 1 penalties	71	22	-49	69.0%
Tier 2 penalties	47	25	-22	46.8%
Tier 3 penalties	302	109	-193	63.9%

The result is important because it validates the project's central design choice: decision quality improved only after the forecast was embedded in the contract logic. The tree-model benchmark demonstrated that several point forecasters were competitive on RMSE and MAE, but those scores alone did not prevent operational breaches. Once the team moved to quantile forecasts and optimized over a 60-day horizon, the policy could hedge against underprediction and steer inventory toward future end-of-month obligations rather than reacting myopically to the next day.

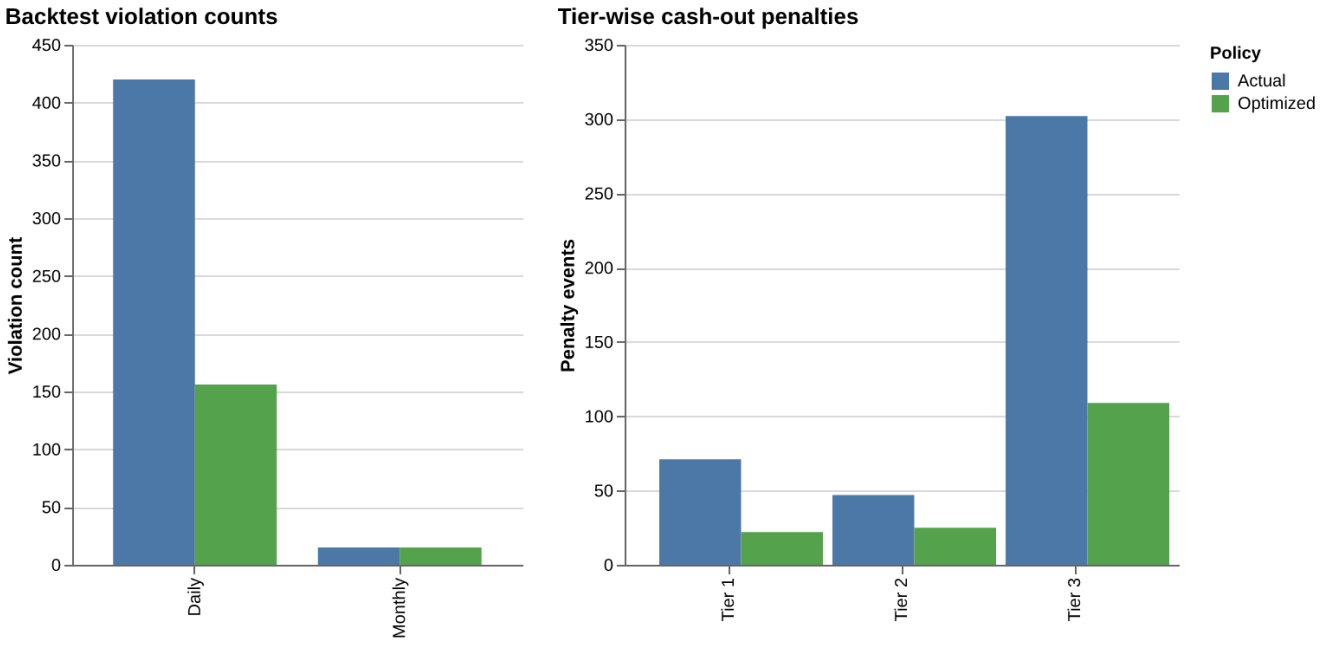

Figure 3: Main results summary from the optimization backtest. The figure consolidates the daily violation reduction, unchanged monthly violation count, and tier-wise penalty improvements that justify the final recommendation.

The EDA also explains why the final model behaved as it did. The series is not only seasonal; it is operationally heterogeneous. Usage 2 dominates scale and noise, Usage 1 carries the clearest weekly cycle, and Usage 2_1 behaves like a persistent auxiliary load. Those facts justify calendar, lag, rolling, and storage-state features rather than a single black-box architecture. They also explain why a simple boosting model remained competitive with newer deep-learning models: the major signal in this problem comes from structured seasonality, operational cadence, and tariff-aware decision rules, not from a hidden representation that only a large neural model can uncover.

The remaining weakness is monthly compliance. The final policy achieved substantial daily gains but did not improve the number of end-of-month violations beyond the actual baseline. Month-end handling remained the hardest part of the project even after adaptive buffers and multi-month constraints were added. For a production deployment, the next design iteration should therefore focus on a direct financial objective that weights daily and monthly breaches by expected cash-out cost, rather than treating all violations as equal events.

5 Conclusions

This project produced a practical forecasting-and-optimization pipeline for Scot Forge rather than a standalone predictive model. The final system respects the operational reality that demand must be forecast with a two-day information lag, converts uncertainty into P10/P50/P95 scenarios, and then chooses delivery nominations with a

rolling linear program that understands daily and end-of-month contract structure. The business result is clear: the optimized policy cut daily violations by 62.9% and materially reduced all three cash-out tiers without worsening month-end compliance.

The project also reinforced a broader consulting lesson: the best model is not the one with the smallest standalone error metric, but the one that produces the best feasible decision under business constraints. Here, that meant moving from a forecasting leaderboard to a robust, quantile-based planning workflow that could absorb volatility instead of underreacting to it.

5.1 Lessons Learned and Suggestions

Two lessons from the project stand out. First, real industrial data is messy, volatile, and often only partially observed, so model design has to respect data latency and operational context rather than assume perfect information. Second, theory does not always win in practice: a relatively simple boosting-based forecast, combined with the right optimization layer, outperformed more complex alternatives because it matched the decision problem more closely. The class-level lesson is similar. Consulting work requires a balance between technical rigor and stakeholder communication, since the client needs KPIs, assumptions, and business implications rather than a purely algorithmic narrative.

For future offerings of the class, the team recommended more structured collaboration with partner organizations, clearer success criteria for sponsored projects, and more transparent inclusion of company feedback in grading. Those suggestions follow directly from the way this project evolved: the most important insights came from stakeholder conversations that changed the objective from prediction to action.

Bibliography

References

Scot Forge. (2026). *Daily Gas Usage and Prediction: Problem Statement* [Course project brief].

Northern Illinois Gas Company. (2025). *Transportation and storage provisions, Ill.C.C. No. 16 – Gas* [Tariff sheets 46–49]. Nicor Gas.

Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *The Annals of Statistics, 29*(5), 1189–1232. https://projecteuclid.org/journals/annals-of-statistics/volume-29/issue-5/Greedy-function-approximation-A-gradient-boosting-machine/10.1214/aos/1013203451.full

scikit-learn developers. (n.d.). *GradientBoostingRegressor*. scikit-learn documentation. Retrieved April 25, 2026, from https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.GradientBoostingRegressor.html

scikit-learn developers. (n.d.). *QuantileRegressor*. scikit-learn documentation. Retrieved April 25, 2026, from https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.QuantileRegressor.html

SciPy Community. (n.d.). *scipy.optimize.linprog(method='highs')*. SciPy documentation. Retrieved April 25, 2026, from https://docs.scipy.org/doc/scipy/reference/optimize.linprog-highs.html

Huangfu, Q., & Hall, J. A. J. (2018). Parallelizing the dual revised simplex method. *Mathematical Programming Computation, 10*(1), 119–142. https://doi.org/10.1007/s12532-017-0130-5

A Appendix: Forecast Benchmark and Visualization Supplement

Figures 4 through 13 preserve the main EDA and optimization visuals generated from the repository so that the body of the report can stay within the page budget.

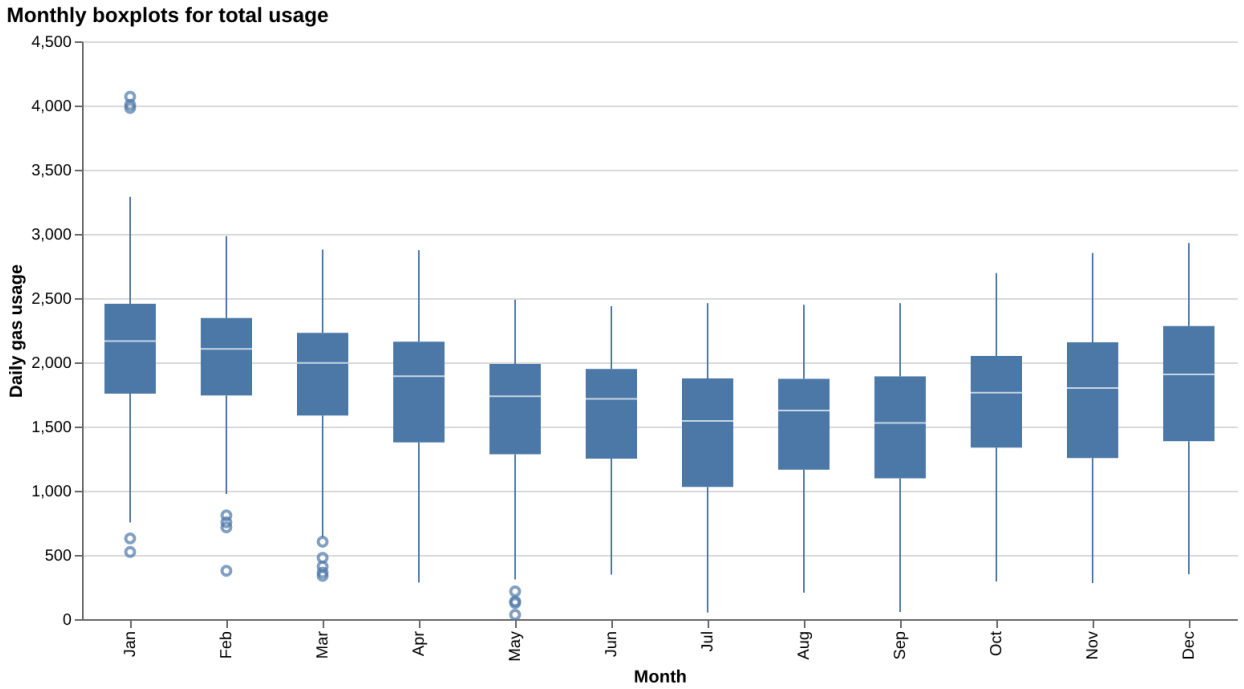

Figure 4: Monthly boxplots for total usage, showing large winter dispersion and tighter summer demand bands.

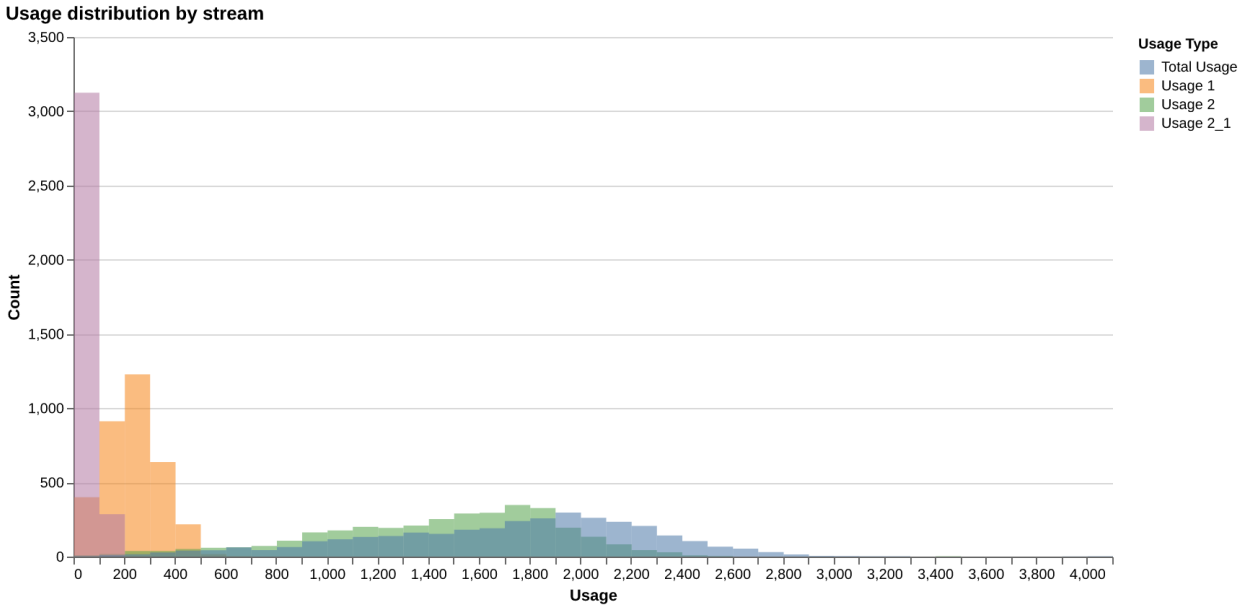

Figure 5: Distribution comparison across the three usage streams and total usage.

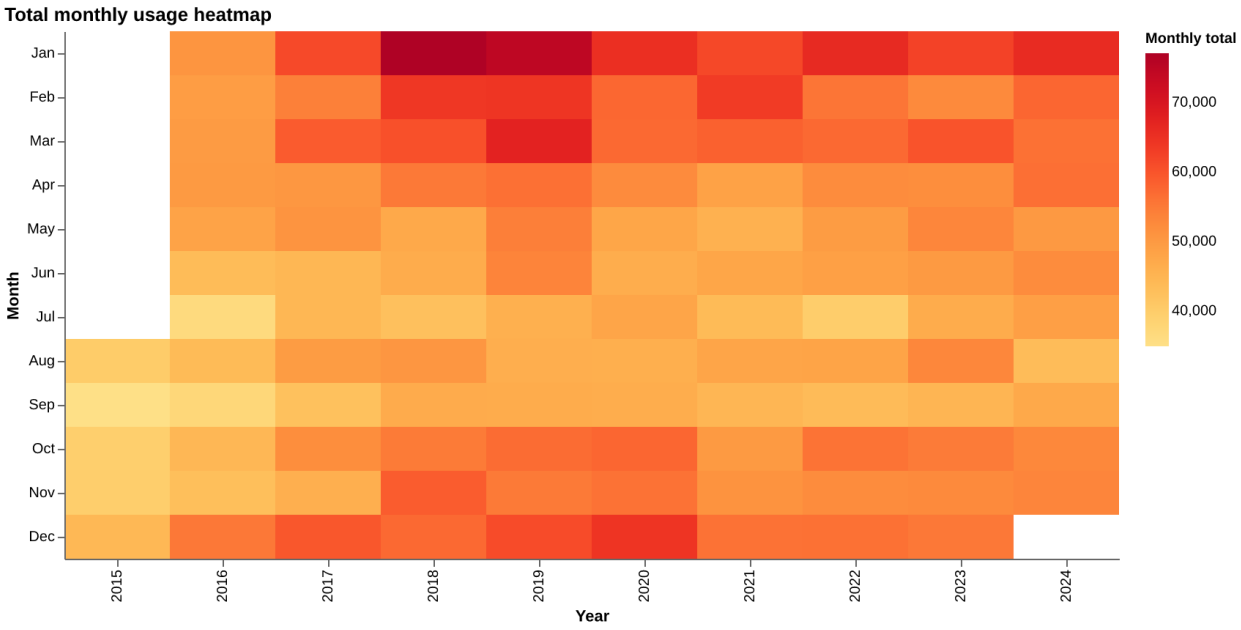

Figure 6: Monthly heatmap for total usage, emphasizing recurring seasonality.

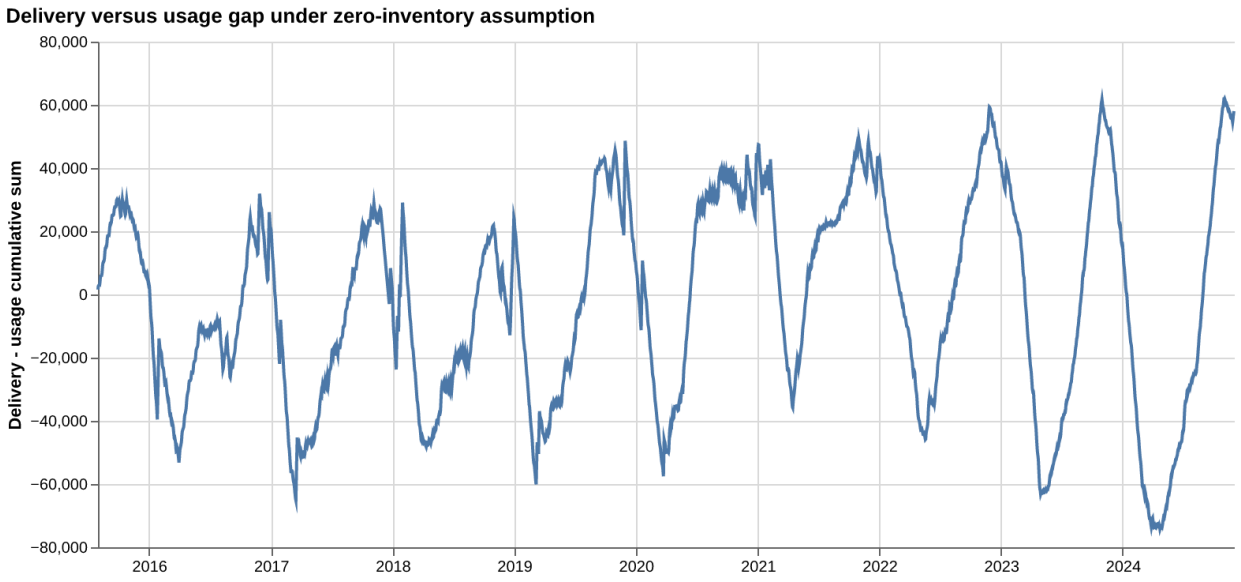

Figure 7: Delivery-versus-usage gap diagnostic that motivated explicit storage-state modeling.

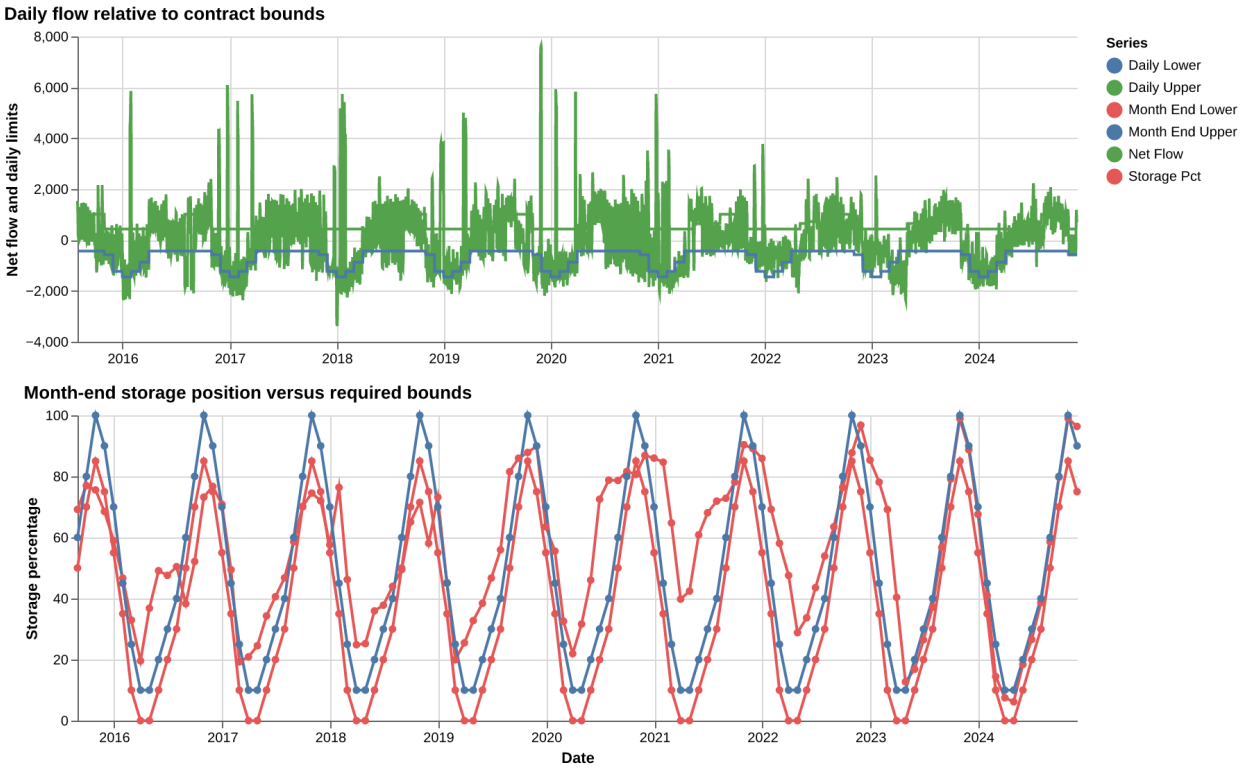

Figure 8: Operational policy dashboard showing daily flow limits and month-end storage bounds.

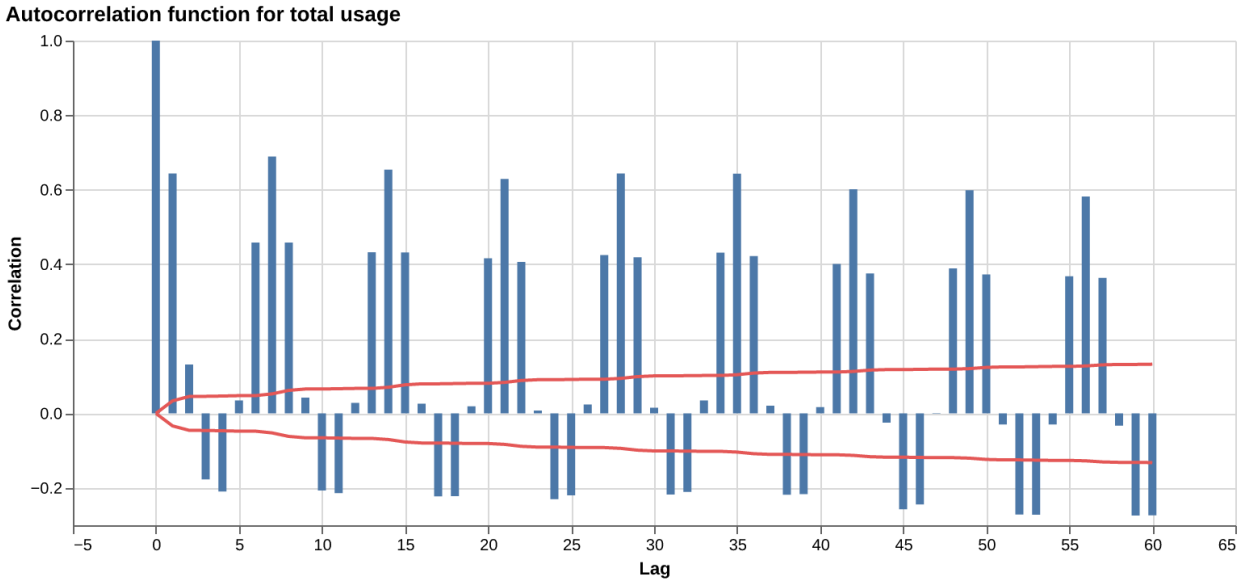

Figure 9: Autocorrelation function for total usage.

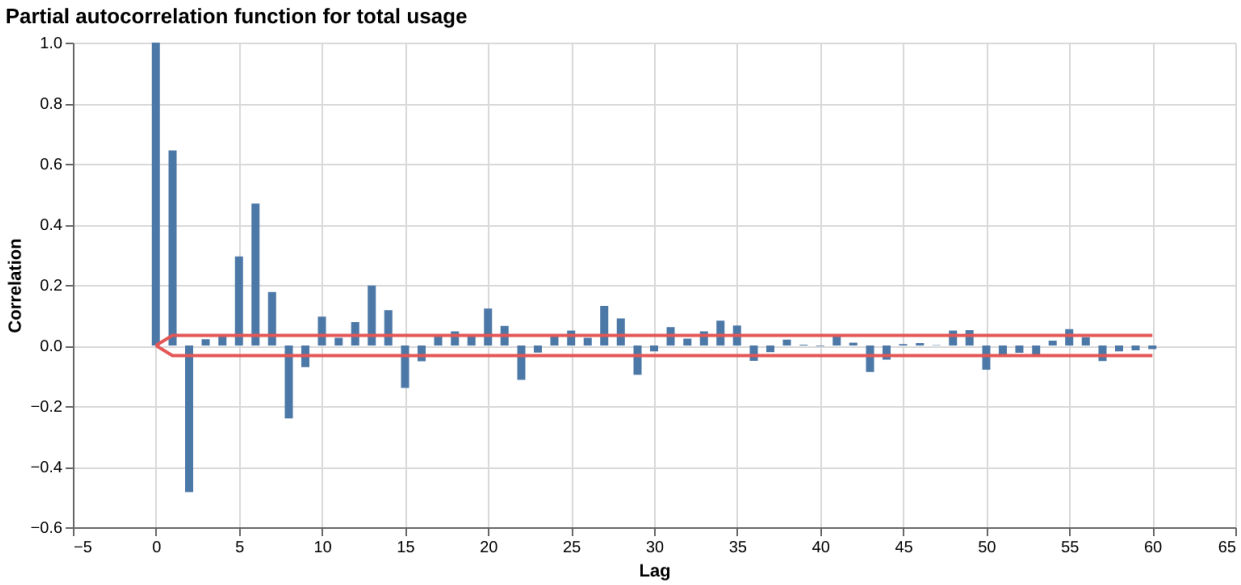

Figure 10: Partial autocorrelation function for total usage.

Figure 11: Daily penalty dashboard: actual vs optimized delivery, flow, and violation days.

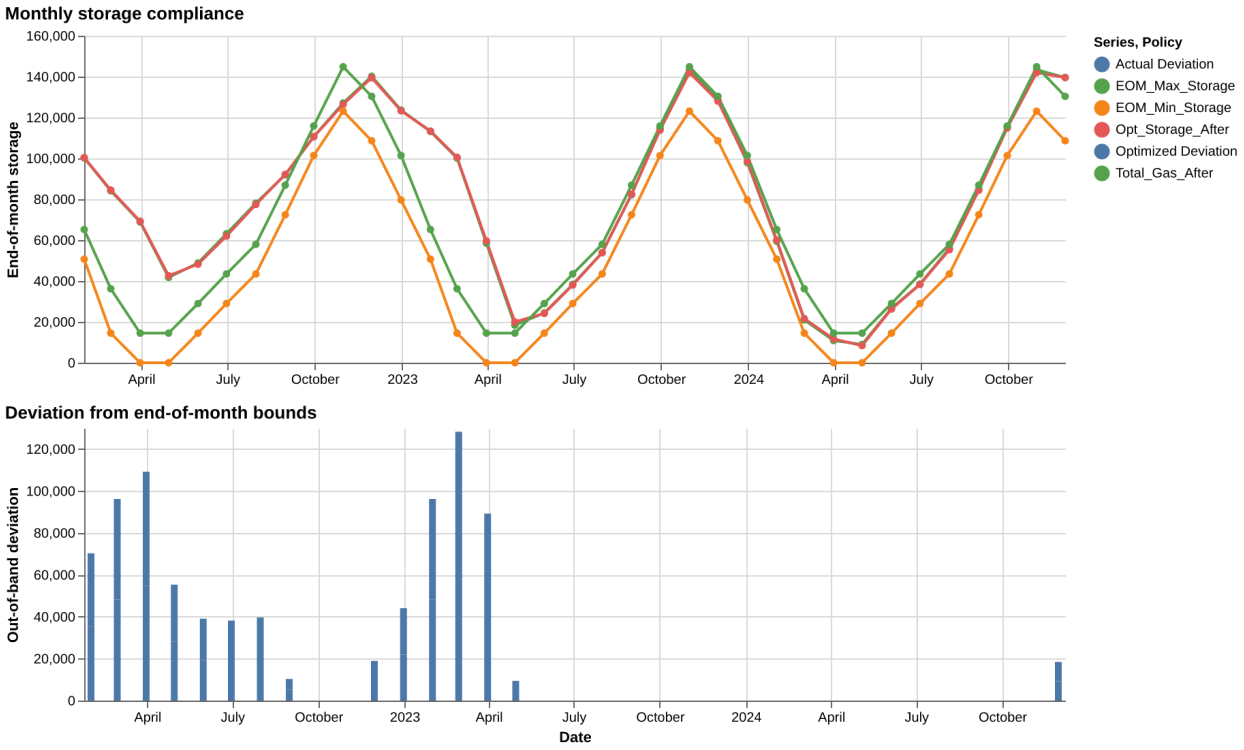

Figure 12: Monthly penalty dashboard: end-of-month storage, limits, and deviations.

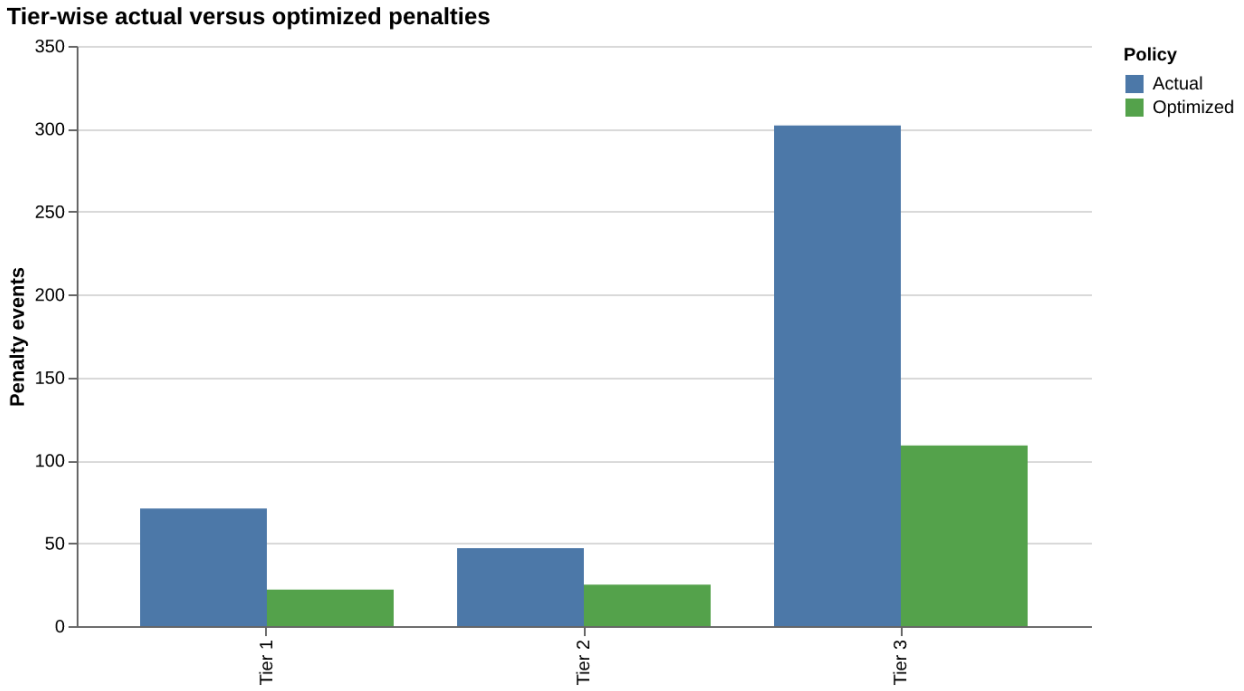

Figure 13: Tier-wise cash-out penalty comparison between actual and optimized operation.

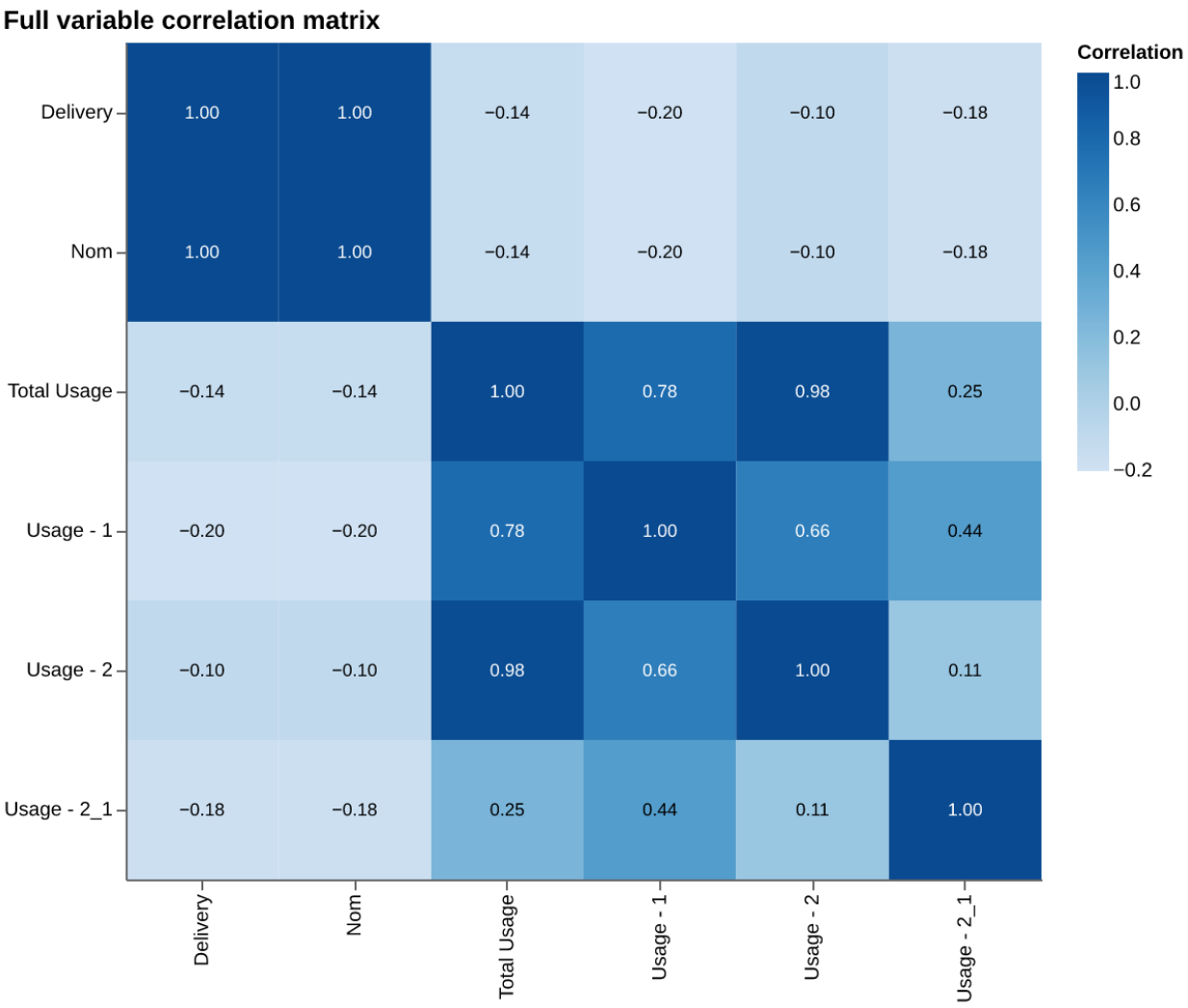

Figure 14: Full variable correlation matrix used during feature selection.